

Sprint Report

Strategic Technology Foresight

Project: AI-Driven Scaling of Strategic Technology Foresight (Digital Innovation Lab, partner: Rohde & Schwarz)

Author: Paul Keck • **Period:** 20–23 June 2026 • **Audience:** Scrum Master (non-technical summary)

1. In a nutshell

This sprint had two big outcomes. First, we found and fixed a hidden quality problem: the futures our AI was generating *looked* varied but were, on closer inspection, close to random noise — and we can now prove, with a measurable test, that the fix produces real substance. Second — and more important — we clarified with the product owner what the project is *really* supposed to deliver and rebuilt the tool to match it: a **reusable engine that works for any topic**, not just the one demo topic. We proved this works by plugging in a completely different subject area and getting sensible results with **zero reprogramming**. Everything is covered by 109 automated tests.

2. What the project is about

Companies like our partner Rohde & Schwarz need to anticipate how technology in their field will evolve over the next decade, so they can decide where to invest. Doing this by hand (“scenario planning”) is slow and only explores a handful of futures. Our project uses AI to do it automatically and at much larger scale: read a body of documents, and produce a structured map of possible futures.

3. Where we started this sprint

A working prototype existed for one test topic (radio-spectrum monitoring). A note from the previous sprint flagged an open concern: the generated futures didn’t seem to fall into clear, distinct groups. The task was simply: **make it better**.

4. What we discovered

When we looked closely, the “futures” the AI produced were essentially **indistinguishable from random** — like shuffling the same deck of cards and calling each shuffle a new strategy.

The root cause is a known weakness of AI language models: they are **relentlessly positive**. When asked “can technology A and technology B work together?”, the AI almost always says “yes”, because with enough effort almost anything *can* be combined. But real strategy is about **trade-offs** — what genuinely conflicts. By never flagging conflicts, the AI produced futures with no real backbone.

To make this concrete and measurable (rather than a matter of opinion), we built a small **litmus test** that compares the AI’s output against pure randomness. This turned a vague worry into a hard, repeatable number — and gave us an objective way to tell whether any later improvement actually worked.

5. What we did

(a) Quality fix — getting honest judgments out of the AI. We changed *how* we ask the AI to weigh technology combinations: instead of “*can* these coexist?” (almost always “yes”), we now ask “*would a single, coherent design really commit to both* — or do they pull in opposite directions?” That reframing makes the AI surface the real trade-offs it was glossing over. The litmus test confirmed the improvement: the output went from “indistinguishable from random” to carrying **real, measurable structure**, and the result held up across repeated runs.

(b) The bigger shift — from one-off prototype to a reusable tool. Mid-sprint we confirmed with the product owner what actually matters to him: he is **not** interested in the spectrum-

monitoring results themselves — that was only a test case. What he wants is a **scalable framework**: feed it any collection of documents (any industry, any topic) and get strategic foresight out, **without anyone having to reprogram it**. This was effectively a scope clarification, and it reshaped our priorities for the rest of the sprint.

The problem: the tool had the demo topic (“spectrum monitoring”, specific company names, a fixed time horizon) **baked into roughly 15 places** in its instructions. Plug in a different topic and you’d get nonsense. So we rebuilt it the right way round. The tool now **reads the documents first, figures out what the topic is on its own**, and feeds that understanding into every downstream step — like turning a single-recipe machine into one that can read and follow *any* recipe. We also added an automatic safeguard that stops anyone from accidentally hard-coding a topic back in.

6. The proof

We docked a brand-new, unrelated body of documents — **autonomous farming / agriculture robots** — and ran the tool. With **no code or wording changes at all**, it correctly recognised the new field and produced a sensible analysis (sensing crops, navigating fields, deciding where to spray, and so on). That is the core promise of the project — “scaling” foresight across topics — demonstrated end to end.

7. Status & quality

- **Done and working**: the topic-detection engine, the quality fix, the full rebuild, and the cross-topic proof.
- **Quality**: 109 automated tests pass, including the safeguard test above.
- **Not yet finalised**: the work is complete but **not yet merged** into the shared codebase — a small, deliberate next step pending a quick review.
- **Honest scope note**: we proved the *analysis core* on the second topic end to end. The final “write-up” steps (polished narrative text and scoring) use the exact same mechanism and are tested, but were not separately re-run on the farming example — and that polished text is the part the product owner has said matters least.

8. Next steps / open decisions

1. **Merge the work** into the shared codebase (small review, then done).
2. **Optional**: update the dashboard/UI, which still shows labels from the old demo topic.
3. **Optional**: run the full end-to-end output on a second topic for a complete showcase.

9. Mini-glossary

- **Scenario / future**: one plausible way the technology landscape could look in about a decade.
- **Framework**: a reusable tool that works for many topics, not a one-off built for a single one.
- **Knowledge base (KB)**: the collection of documents we feed in as the tool’s source material.
- **Trade-off**: a real conflict where choosing one option costs you another — the substance that makes a strategy meaningful.